AI WEATHERWISE

Intelligent Weather Forecasts and AI-Powered Weather Insights

---

PHASE 2
REQUIREMENT ANALYSIS

System Requirements Documentation

Document Date: September 28, 2026

Status: Requirements Analysis Phase

BSc Computer Science Final Year Project

REST API Backend | Node.js | Express.js | MongoDB | Mongoose | JWT | Google Gemini AI

Table of Contents

1. Functional Requirements **4**

1.1 User Registration 4

1.2 User Login 4

1.3 Protected User Access 5

1.4 Weather Information Retrieval 5

1.5 Favorite Location Management 6

1.6 AI Weather Summary 7

1.7 AI Weather Recommendations / Insights 7

1.8 API Response Handling 8

1.9 Validation and Error Handling 8

1.10 Functional Requirements Table 9

2. Non-Functional Requirements **11**

3. User Roles and Use Cases **12**

3.1 System Roles 12

3.2 Authenticated User Capabilities 12

3.3 Use Cases 13

4. System Architecture Overview **14**

4.1 Architecture Description 14

4.2 Architecture Layers 14

4.3 Architecture Diagram 15

4.4 Key Architectural Principles 15

4.5 Requirements Traceability 16

Note: right-click the table above and choose "Update Field" if page numbers do not appear after opening the file.

Document Scope and Conventions

This document is the Phase 2 – Requirement Analysis deliverable for AI WeatherWise, an intelligent weather-focused REST API backend developed as a BSc Computer Science final-year project. It states what the system must do, the quality attributes it must satisfy, who interacts with it, and how the architecture supports those needs.

Source Basis

The requirements are derived from the AI WeatherWise Professional Project Documentation, which is the primary source for the intended system. Statements about what the backend currently does are drawn from the documented implementation and the API testing evidence recorded in that documentation. The FitTrack AI Phase 2 document was used only as a model for structure and presentation.

Wording Conventions

- **"The system must …"** states a requirement defined for AI WeatherWise.
- **"The backend currently …"** states functionality that exists in the implemented backend.
- **"The project documentation specifies …"** refers to design described in the WeatherWise project documentation.

Implemented versus Specified

Not every requirement in this document has recorded test evidence. The documentation confirms four tested API calls: user registration, user login, current-weather retrieval, and AI weather recommendation. Favorite-location CRUD, protected-route enforcement, and AI weather summaries are specified in the documented design; where an endpoint path, status code, or test result is not stated in the source, this document says so instead of inventing it. The Implementation Status column in Section 4.5 records this for each requirement.

Backend Environment

The AI WeatherWise backend runs on PORT 5001, configured through the PORT environment variable together with MONGO_URI, JWT_SECRET, and GEMINI_API_KEY. Data is stored in MongoDB Atlas. Endpoints were exercised with an API client during development (the project documentation records Postman as the testing tool; Thunder Client was also used during backend development).

1. Functional Requirements

Functional requirements define what the AI WeatherWise system must do. These requirements describe the specific features, functions, and behaviors that allow users to register securely, retrieve current weather, manage favorite locations, and receive AI-generated weather summaries and recommendations through the REST API.

1.1 User Registration

The system must allow new users to create an account. A user registers by providing a name, an email address, and a password. The system must ensure that each email address is unique and must reject an attempt to register an email that already exists. Passwords must be hashed with bcrypt.js before being stored, so that a plain-text password is never written to the database.

What the system must do	Create a new user account from submitted registration details.
Input	name, email, password (sent to POST /api/auth/register).
Processing	Validate that name, email and password are present. Check the email against a standard email-format pattern; store it in lowercase; enforce uniqueness. Enforce a minimum password length of 6 characters. Hash the password with bcrypt.js and save the User document (with createdAt / updatedAt timestamps) in MongoDB.
Output	HTTP 201 Created with the user object and a JWT (as observed in the recorded API test).
Authentication	Not required (public endpoint).
Why it is needed	Registration establishes the identity to which favorite locations are attached and enables secure access to personalized features.
Status	The backend currently implements registration; the 201 Created result is confirmed by the recorded API test.

1.2 User Login

Registered users must be able to authenticate with their email and password. The system must look up the account, compare the submitted password with the stored hash using bcrypt, and, when the credentials are valid, generate a signed JSON Web Token (JWT). This token must be included in subsequent requests to protected endpoints so that the system can identify the caller.

What the system must do	Authenticate a registered user and issue a JWT.
Input	email, password (sent to POST /api/auth/login).

Processing	Look up the user by email in MongoDB. Verify the password using bcrypt.compare against the stored hash. On success, sign a JWT using the JWT_SECRET environment variable. On failure, reject the request through the centralized error-handling mechanism.
Output	HTTP 200 OK with the user object and a JWT (as observed in the recorded API test).
Authentication	Not required to call the login endpoint; the JWT it returns is required by protected endpoints.
Why it is needed	Login converts a registered account into a verifiable session token without the server storing session state.
Status	The backend currently implements login; the 200 OK result is confirmed by the recorded API test.

1.3 Protected User Access

The system must restrict personalized functionality to authenticated users. Authentication middleware must read the Bearer token from the request's Authorization header, verify it, and only then allow the request to reach a protected route. Requests with a missing or invalid token must be refused. Once identified, the user's identity is used to scope data so that each user can only work with their own favorite locations.

What the system must do	Allow access to protected routes only when a valid JWT is presented, and keep user data scoped to its owner.
Input	Authorization header in the form "Bearer <token>".
Processing	authMiddleware.js extracts and verifies the JWT. A valid token identifies the authenticated user for the controller. An invalid or absent token is rejected before controller logic runs.
Output	Access granted to the requested protected route, or an error response when authentication fails.
Authentication	Required for protected routes such as favorite-location management. Weather lookup is a public route and does not require a token.
Why it is needed	It separates public from private functionality and protects user-specific data.
Status	Authentication middleware is part of the documented backend structure (middleware/authMiddleware.js). The source does not record a negative-path test (for example, a request with a missing token).

Scope note: the documentation states that personalized features remain protected behind authentication and lists AI interaction among the registered user's activities, but its tested evidence does not explicitly confirm whether POST /api/ai/weather-recommendation enforces a JWT. This document therefore specifies authenticated access for AI features as the intended design and flags the point for confirmation during later phases.

1.4 Weather Information Retrieval

The system must retrieve current weather information for a city requested by the client. The project documentation specifies the OpenWeatherMap API as the external source of live weather data. The Weather Controller receives the request, the Weather Service calls the external API, and the result is returned as a structured JSON response. This endpoint is public, so casual users can check conditions without an account.

What the system must do	Return current weather metrics for the requested city.
Input	city, supplied as a URL parameter (GET /api/weather/:city).
Processing	Weather Controller passes the city to the Weather Service. Weather Service requests current conditions from OpenWeatherMap. If an external weather API key is not configured, the service returns a structured mock response flagged with an isMock indicator so that the API remains usable during development and testing.
Output	HTTP 200 OK with city, temperature, humidity, windSpeed and condition (plus an isMock flag when fallback data is used).
Authentication	Not required (public route).
Why it is needed	Current weather is the base data for every other feature, including the AI summary and recommendation.
Status	The backend currently implements this endpoint; the 200 OK result is confirmed by the recorded API test.

1.5 Favorite Location Management

The system must let an authenticated user maintain a personal list of favorite weather locations. The project documentation specifies a Location Controller performing create, read, update and delete operations, routed through locationRoutes.js. Each location is stored with a city, a country, and a reference to its owning user, giving a one-to-many relationship: one User owns many favorite Locations.

What the system must do	Add a favorite location (city, country). View the authenticated user's saved locations. Update an existing saved location. Delete a saved location.
Input	city and country when adding; updated fields when editing. The owning user is taken from the authenticated token, not from user input.

Processing	Verify the JWT. On create, populate the user field with the authenticated user's ObjectId and validate the required fields through the Mongoose schema. On read, return only locations belonging to that user. On update and delete, perform an ownership check before modifying MongoDB. A compound unique index on user, city and country prevents the same user from saving the same city and country twice.
Output	The created, retrieved, updated or deleted location data, or a confirmation, returned as JSON.
Authentication	Required for every location operation.
Why it is needed	It addresses the documented problem of tracking several favorite cities securely and keeping them tied to one user.
Status	The Location model, controller and routes are part of the documented backend structure. The source does not state the endpoint paths and records no test results for these operations; this document does not assume either.

1.6 AI Weather Summary

The system must be able to turn raw weather metrics into a natural-language summary. The weather details (city, temperature, humidity and condition) are passed to the AI Service, which calls Google Gemini through the @google/genai SDK and returns a readable narrative. A summary is more useful than raw numbers because most users want to know what conditions mean rather than interpret individual metrics; this is one of the problems named in the project's problem statement.

What the system must do	Produce a natural-language summary of the supplied weather conditions.
Input	city, temperature, humidity, condition.
Processing	AI Controller → Gemini Service (@google/genai) → Google Gemini AI → summary text.
Output	A JSON response containing the AI-generated summary.
Authentication	Specified as an authenticated (personalized) feature; see the scope note in Section 1.3.
Why it is needed	It removes the need for the user to interpret raw metrics.
Status	The project documentation describes summary generation as part of the AI Service integration and presents its workflow as conceptual. A separately tested summary endpoint is not shown in the source.

1.7 AI Weather Recommendations / Insights

The system must generate a weather-related recommendation using Google Gemini AI. The client sends the current weather details to the AI endpoint, the request passes through the AI Controller and Gemini Service, and Gemini returns a condition-aware suggestion such as clothing or hydration advice. The documentation refers to these as personalized recommendations; the inputs it defines are weather

details only (city, temperature, humidity, condition), so the personalization is driven by conditions rather than by a stored user profile.

What the system must do	Return a practical, condition-aware recommendation for the supplied weather.
Input	city, temperature, humidity, condition (sent to POST /api/ai/weather-recommendation).
Processing	AI Controller → Gemini Service → Google Gemini AI → recommendation text.
Output	HTTP 200 OK with a Gemini-generated recommendation (the documentation's example is advice to stay hydrated and wear light cotton clothes).
Authentication	Not explicitly confirmed in the tested evidence; specified as an authenticated feature (see Section 1.3).
Why it is needed	It converts weather data into an action the user can take.
Status	The backend currently implements this endpoint; the 200 OK result is confirmed by the recorded API test.

The three levels of information the system deals with are distinct:

Level	What it is	Example source in WeatherWise
Raw weather information	Measured values returned by the weather service.	temperature, humidity, windSpeed, condition from GET /api/weather/:city
AI-generated interpretation	A plain-language description of what the conditions are like.	Natural-language summary produced by Gemini (Section 1.6)
Recommendation / insight	Advice on what to do given the conditions.	Output of POST /api/ai/weather-recommendation (Section 1.7)

1.8 API Response Handling

All API endpoints must return structured JSON responses with a consistent format. Responses must carry an appropriate HTTP status code and a clear result. The recorded tests confirm HTTP 201 Created for registration and HTTP 200 OK for login, weather retrieval and AI recommendation. The documentation does not list specific status codes for failure scenarios; these are produced by the centralized error-handling mechanism, and the system must ensure that they follow standard HTTP semantics.

Success responses	JSON body containing the data payload for the operation (for example, user object and JWT, weather metrics, or AI text).
Error responses	JSON body describing the failure, with a status code chosen by the centralized error handler.
HTTP status codes	201 Created (registration) and 200 OK (login, weather, AI recommendation) are confirmed by the source.
Why it is needed	A uniform format lets any client consume the API without endpoint-specific parsing.

1.9 Validation and Error Handling

The system must validate input before processing it and must handle failures without crashing or exposing internal details. The project documentation confirms two project-wide mechanisms: centralized error handling and request sanitization. The validation rules below are those stated in the documentation and schemas.

Validation rules

- Registration: name is required; email is required, unique, lowercase, and must match a standard email pattern; password is required and at least 6 characters.
- Favorite location: city and country are required; the same user cannot save the same city and country combination twice.
- Incoming request data is sanitized as part of request processing.

Failure handling

Failure type	Required system behavior	Source support
Invalid input	Reject the request with a clear error message; do not create or change data.	Schema validation and centralized error handling documented.
Missing required data	Reject the request and indicate which required field is missing.	Required fields documented for users and locations.
Authentication failure	Refuse access to protected routes when the token is missing or invalid, and refuse login for wrong credentials.	Authentication middleware and bcrypt comparison documented; specific status codes not stated.
External weather API failure	Keep the endpoint responsive by returning a structured fallback (mock) response flagged with isMock.	Fallback mode documented for the case where the weather API key is not configured.
AI service failure	Return an error through the centralized handler rather than failing silently. The documentation lists dependence on Gemini availability as a limitation and describes no fallback text.	Limitation documented; no fallback mechanism documented.
Database error	Surface a controlled error through the centralized handler without exposing connection details.	Centralized error handling documented; database-specific behavior not detailed.

1.10 Functional Requirements Table

Priorities are assigned by this analysis from the project objectives: High for functions on which the core objectives depend, Medium for supporting functions.

Req ID	Requirement	Description	Priority
FR-001	User Registration	Users create an account with name, email and password. Email must be unique and valid; password must be at least 6 characters and hashed with bcrypt.js before storage.	High

Req ID	Requirement	Description	Priority
FR-002	User Login	Users log in with email and password. The system verifies credentials with bcrypt and returns a JWT on success.	High
FR-003	Protected API Access	Protected routes require a valid Bearer JWT verified by authentication middleware. User data is scoped to the authenticated user.	High
FR-004	Weather Information Retrieval	Any client can request current weather for a city (GET /api/weather/:city). Data comes from OpenWeatherMap; a flagged mock response is returned when the API key is not configured.	High
FR-005	Add Favorite Location	Authenticated users save a favorite location (city, country) linked to their account. Duplicate city/country pairs per user are rejected.	High
FR-006	View Favorite Locations	Authenticated users retrieve their own saved favorite locations only.	High
FR-007	Update Favorite Location	Authenticated users update an existing favorite location they own.	Medium
FR-008	Delete Favorite Location	Authenticated users delete a favorite location they own.	Medium
FR-009	AI Weather Summary	System sends weather details (city, temperature, humidity, condition) to Gemini and returns a natural-language summary.	Medium
FR-010	AI Weather Recommendation	System sends weather details to Gemini through POST /api/ai/weather-recommendation and returns a condition-aware recommendation.	High
FR-011	API Response Handling	All endpoints return structured JSON with appropriate HTTP status codes (201 for registration, 200 for successful retrieval) and clear results.	High
FR-012	Validation and Error Handling	Input is validated and sanitized before processing; failures are handled by centralized error handling with clear messages.	High

2. Non-Functional Requirements

Non-functional requirements describe how the AI WeatherWise system should perform and the quality attributes expected from it. They cover security, reliability, performance, maintainability and related qualities, and apply across all functional requirements.

Requirement	Description
Security	User accounts and personalized data must be protected from unauthorized access. Protected routes require a valid JWT, passwords are hashed, and sensitive configuration (JWT_SECRET, MONGO_URI, GEMINI_API_KEY) is kept in environment variables, not source code. CORS is managed at the server.
Authentication	JWT must be used for stateless authentication. Protected endpoints require a valid Bearer token, and the token must be verified by middleware on each protected request. Token lifetime should be limited to an appropriate period.
Password Protection	Passwords must be hashed with bcrypt.js before storage. Plain-text passwords must never be stored. A minimum length of 6 characters is enforced before hashing.
Data Privacy	Favorite locations belong to the user who created them. Each location stores its owning user, and read, update and delete operations must be limited to the authenticated owner. Stored password data is only the hash.
Performance	Weather retrieval, database operations and AI requests must complete within a reasonable time. Weather and AI calls depend on external services and are expected to be slower than database-only calls, but must not block the server from handling other requests.
Reliability	The weather endpoint must keep returning a structured response through mock fallback mode when the weather API key is not configured. Database, weather and Gemini failures must be reported through centralized error handling rather than causing the server to fail.
Maintainability	The code must follow the MVC-style structure documented for the project, with separate folders for config, controllers, middleware, models, routes and services. Each layer should have one clear responsibility so that features can be changed without side effects.
Scalability	The REST design supports adding endpoints and clients without restructuring. JWT authentication is stateless, so the server keeps no session state, and MongoDB document storage can accommodate growth in users and saved locations.
Usability	Endpoints must be easy to understand and follow REST conventions, with public routes (weather) clearly separated from protected routes. Responses must be readable JSON, and the AI output must be plain natural language.
Input Validation	All user input must be validated before processing: required fields, email format, minimum password length, required city and country, and duplicate-location prevention. Incoming request data must be sanitized.
Error Handling	Errors must be handled at a central point and returned as meaningful messages. Responses must not expose secrets, stack traces, connection strings or other internal details.
API Consistency	Endpoints must use consistent naming (for example the /api/auth, /api/weather and /api/ai prefixes), correct HTTP methods (GET, POST, PUT, DELETE), standard status codes, and a uniform JSON response structure.

3. User Roles and Use Cases

3.1 System Roles

Weather Application User (Public)

An individual who uses the API without creating an account. The project documentation describes this user as a “weather application user” who can call the public current-weather endpoint directly. This user has no access to personalized or protected functionality.

Authenticated User

The primary user of AI WeatherWise. An Authenticated User has registered an account and logged in, and holds a JWT that is sent as a Bearer token. This user can access protected functionality, including favorite-location management, and can request AI weather summaries and recommendations. Their saved locations are scoped to their own account.

Admin

The project documentation names an Admin role with oversight responsibilities: monitoring API health and system logs, managing database integrity, and account-level oversight of registered users. The documentation does not define admin-specific endpoints or an admin interface, so this document treats the role as operational oversight. No admin functional requirements or use cases are specified in Phase 2.

3.2 Authenticated User Capabilities

Register and Create Account

Create a new account using name, email and password.

Login to System

Authenticate with email and password to receive a JWT.

Access Protected APIs

Send the JWT as a Bearer token to use protected routes.

Retrieve Weather Information

Request current weather for a city. This capability is also available to unauthenticated users, because the weather endpoint is public.

Manage Favorite Locations

Add, view, update and delete personal favorite locations (city and country).

Request AI Weather Summaries

Submit weather details to receive a natural-language summary generated by Google Gemini.

Request AI Weather Recommendations

Submit weather details to receive a condition-aware recommendation, such as clothing or hydration advice.

3.3 Use Cases

ID	Use Case	Actor	Description	Expected Result
UC-001	Register New User	Weather Application User	User submits name, email and password. System validates the fields, checks email uniqueness, hashes the password and stores the user.	Account created; password hashed; 201 Created with user object and JWT.
UC-002	Login User	Registered User	User submits email and password. System finds the account and verifies the password with bcrypt.	Credentials validated; JWT issued; 200 OK.
UC-003	Access Protected API	Authenticated User	User sends a request with a Bearer JWT to a protected route. Middleware verifies the token before the controller runs.	Valid token: access granted. Missing or invalid token: request refused.
UC-004	Retrieve Weather Information	Weather Application User / Authenticated User	User requests weather for a city. System calls OpenWeatherMap through the Weather Service, or returns flagged mock data if the API key is not configured.	200 OK with city, temperature, humidity, windSpeed, condition (isMock when fallback).
UC-005	Add Favorite Location	Authenticated User	User submits city and country. System verifies the JWT, attaches the user's ID and enforces the unique user/city/country index.	Location saved for the user; duplicates rejected.
UC-006	View Favorite Locations	Authenticated User	User requests their saved locations. System returns only locations owned by the authenticated user.	User's locations returned as JSON.
UC-007	Update Favorite Location	Authenticated User	User submits changes to a saved location. System checks ownership, then updates the record.	Location updated; confirmation returned.
UC-008	Delete Favorite Location	Authenticated User	User removes a saved location. System checks ownership, then deletes the record.	Location deleted; confirmation returned.
UC-009	Request AI Weather Summary	Authenticated User	User submits weather details. AI Controller passes them to the Gemini Service, which asks Google Gemini for a summary.	Natural-language weather summary returned as JSON.
UC-010	Request AI Weather Recommendation	Authenticated User	User submits city, temperature, humidity and condition to POST /api/ai/weather-recommendation. Gemini generates advice.	200 OK with a condition-aware recommendation.

4. System Architecture Overview

4.1 Architecture Description

AI WeatherWise is a REST API backend built with Node.js and Express.js. The project documentation describes a layered architecture that follows an MVC-style pattern. Because the system is a headless backend with no template-based user interface, the “view” role is played by the routing layer, which maps each request path to a controller. A request passes through the following chain:

Client → Express.js REST API → Routes → Middleware → Controllers → Services → Models (Mongoose) → MongoDB Atlas

Two external integrations sit behind the service layer: the OpenWeatherMap API for current weather data and Google Gemini AI (through @google/genai) for summaries and recommendations. The Express server, which currently runs on PORT 5001, loads its configuration from environment variables, connects to MongoDB Atlas through Mongoose, and handles JSON parsing, CORS and centralized error handling. Authentication and location requests are handled by controllers working with the Mongoose models; weather and AI requests are delegated to the services.

4.2 Architecture Layers

Presentation / API Layer

Express.js routes (authRoutes.js, locationRoutes.js, weatherRoutes.js, aiRoutes.js) expose the REST endpoints and map HTTP methods to controllers. All responses are JSON.

Authentication & Security Layer

authMiddleware.js verifies Bearer JWTs on protected routes and separates them from public routes. Passwords are hashed with bcrypt.js, request data is sanitized, CORS is managed at the server, and secrets are kept in the .env file.

Business Logic / Controller Layer

The Auth, Location, Weather and AI controllers receive requests, apply constraints, delegate to models or services, and return standardized JSON responses. They contain the rules for registration, login, favorite-location ownership, and orchestration of weather and AI requests.

Data Access Layer (Mongoose / MongoDB Atlas)

Mongoose schemas for User and Location define fields, validation, timestamps and the unique user/city/country index. Data is persisted in the Users and Locations collections in MongoDB Atlas, connected through MONGO_URI in config/db.js.

External Integration Layer

The Weather Service communicates with OpenWeatherMap, and the AI Service communicates with Google Gemini AI through the @google/genai SDK. These services isolate third-party calls from controllers, so external providers can be changed or mocked without changing controller logic.

4.3 Architecture Diagram

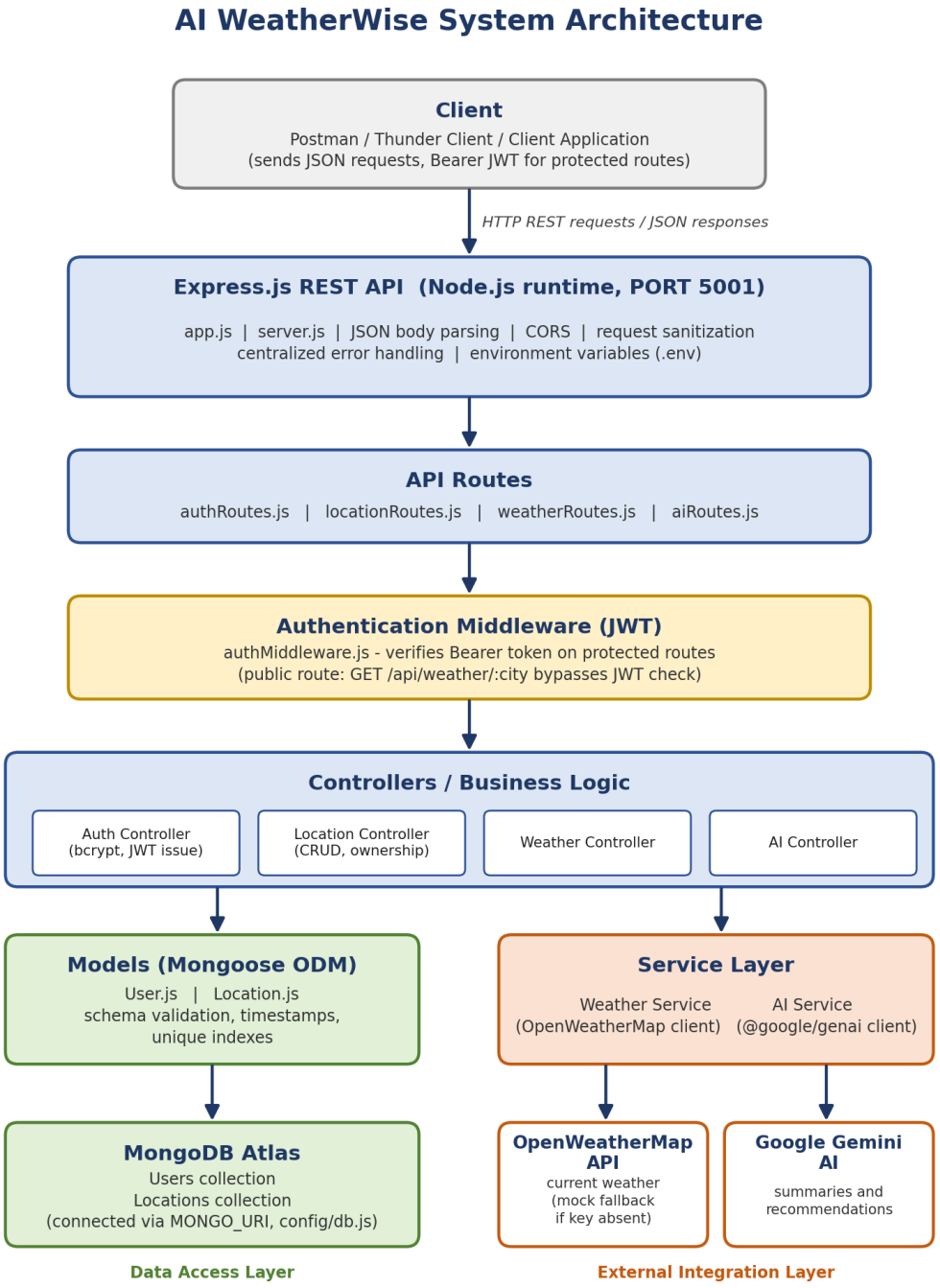


Figure 4.1: AI WeatherWise System Architecture

4.4 Key Architectural Principles

Separation of Concerns

Routes, middleware, controllers, services and models each have a distinct role, and the source folders mirror this separation. Changing the weather provider, for example, affects only the Weather Service.

RESTful Design

Endpoints use standard HTTP methods and status codes and are grouped by feature: /api/auth, /api/weather and /api/ai for the documented endpoints, plus a separate route group for favorite locations.

Security First

JWT authentication, bcrypt hashing, public/private route separation, user-scoped data, request sanitization and environment-based secrets are applied throughout.

Scalability

Stateless JWT authentication and MongoDB document storage allow the system to grow in users and saved locations without a change to the architecture.

Maintainability

A consistent MVC-style folder structure keeps the codebase understandable and makes future features such as new endpoints straightforward to add.

External Service Integration

OpenWeatherMap and Google Gemini AI are accessed only through dedicated services. The mock fallback for weather data shows the intent to keep the API usable when an external dependency is unavailable.

4.5 Requirements Traceability

Phase 2 – Requirement Analysis documents the functional and non-functional requirements of AI WeatherWise, together with its roles and use cases. These form the foundation for design, implementation and testing in later phases. Each functional requirement below is mapped to the use case that exercises it, and its current status is recorded honestly against the source evidence. Requirements FR-011 and FR-012 are cross-cutting: they apply to every use case and therefore have no single use case.

Req ID	Requirement	Use Case	Implementation Status (per source)
FR-001	User Registration	UC-001	Implemented; registration test recorded (201 Created).
FR-002	User Login	UC-002	Implemented; login test recorded (200 OK).
FR-003	Protected API Access	UC-003	Authentication middleware documented; no negative-path test recorded.
FR-004	Weather Information Retrieval	UC-004	Implemented; weather test recorded (200 OK, city, temperature, humidity, windSpeed, condition).
FR-005	Add Favorite Location	UC-005	Documented design (Location Controller, model, routes); endpoint path and test evidence not stated in source.
FR-006	View Favorite Locations	UC-006	Documented design; endpoint path and test evidence not stated in source.

Req ID	Requirement	Use Case	Implementation Status (per source)
FR-007	Update Favorite Location	UC-007	Documented design; endpoint path and test evidence not stated in source.
FR-008	Delete Favorite Location	UC-008	Documented design; endpoint path and test evidence not stated in source.
FR-009	AI Weather Summary	UC-009	Described as part of the AI Service; workflow conceptual; separate endpoint not shown in source.
FR-010	AI Weather Recommendation	UC-010	Implemented; AI recommendation test recorded (200 OK).
FR-011	API Response Handling	All	201 and 200 responses confirmed for tested endpoints; failure codes not specified in source.
FR-012	Validation and Error Handling	All	Validation rules and centralized error handling documented; negative-path results not recorded.

Not every non-functional requirement maps to one use case. They are satisfied through the architecture in Section 4: security and authentication through the middleware and bcrypt hashing, maintainability through the MVC-style structure, and reliability through the weather fallback and centralized error handling.

The entities shown in the project's conceptual data model but not present in the current schemas (weather history, stored AI insights, alerts, subscriptions, API usage logs and AI model records) are outside the scope of these Phase 2 requirements.